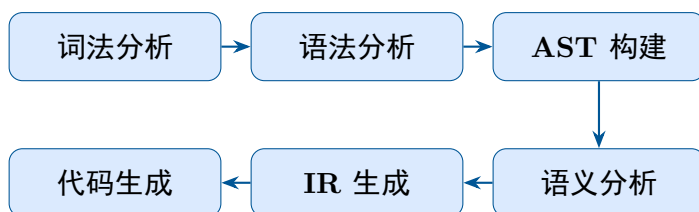


简易编译器 设计指导手册

编译原理课程大作业参考文档



适用读者 编译原理课程本科生

实现语言 C++17

构建工具 CMake

文档目的 提供设计思路，不提供完整代码

目录

1 项目概述	3
1.1 整体编译流程	3
1.2 目标语言特性（建议支持）	3
2 项目文件结构	4
2.1 各模块职责一览	5
3 辅助工具链	5
3.1 构建系统：CMake	6
3.1.1 常用构建命令	6
3.2 词法/语法生成器：Flex & Bison（可选）	7
3.3 单元测试：Google Test	7
3.4 调试工具：GDB	7
4 编译器各阶段详解	8
4.1 阶段一：词法分析（Lexer）	8
4.1.1 任务	8
4.1.2 Token 的设计	8
4.1.3 实现思路	9
4.2 阶段二：语法分析（Parser）	9
4.2.1 任务	9
4.2.2 文法定义示例（EBNF）	9
4.2.3 递归下降解析器	10
4.2.4 错误恢复：Panic Mode（恐慌模式）	10
4.3 阶段三：抽象语法树（AST）	11
4.3.1 什么是 AST	11
4.3.2 节点类设计（C++ 类层次）	11
4.3.3 访问者模式（Visitor Pattern）	12
4.4 阶段四：语义分析	13

4.4.1	任务	13
4.4.2	符号表设计	13
4.4.3	类型检查	14
4.5	阶段五：中间代码生成（IR）	14
4.5.1	为什么需要中间表示	14
4.5.2	三地址码（Three-Address Code）	14
4.5.3	DAG（有向无环图）	15
4.6	阶段六：代码生成	16
4.6.1	目标选择（由易到难）	16
5	完整构建步骤	16
5.1	开发环境准备	16
5.2	推荐开发顺序	17
5.3	增量验证策略	17
6	性能优化建议	18
6.1	常量折叠（Constant Folding）	18
6.2	代数化简（Algebraic Simplification）	18
6.3	负号合并详细示例	19
6.4	公共子表达式消除（CSE）	19
6.5	死代码消除（Dead Code Elimination）	20
6.6	指令重排（Instruction Reordering）	20
7	实战难点预警	21
7.1	内存管理	21
7.2	统一错误处理	21
7.3	常见 Bug 预警	22
7.4	推荐学习资源	23
8	总结	23

1 项目概述

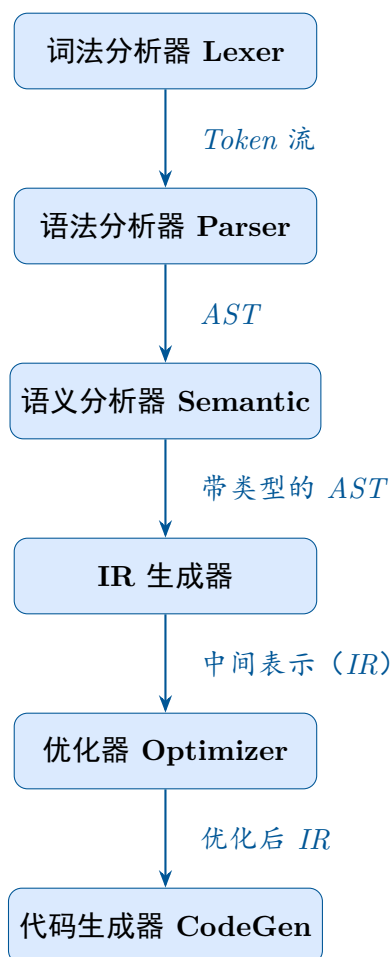
本手册旨在指导大学生从零开始，使用 C++ 语言构建一个简易编译器。编译器将源代码翻译为目标代码，内部清晰地划分为多个流水线阶段，每个阶段各司其职。

本文档定位

本文档提供设计思路、数据结构建议和模块划分方向，不提供可直接运行的完整代码。你需要根据这些思路自己动手实现——这才是真正学习的方式。

1.1 整体编译流程

源代码经过各阶段处理，最终生成可执行代码：



1.2 目标语言特性（建议支持）

- 基本数据类型：整数 `int`、布尔值 `bool`

- 算术运算符：+、-、*、/
- 比较运算符：==、!=、<、<=、>、>=
- 控制流：if-else、while
- 变量声明与赋值
- 代码块 {}（用于作用域）
- （可选进阶）函数定义与调用

2 项目文件结构

合理的文件结构是工程质量的基础，便于模块化开发和后期维护。

```

1 MyCompiler/
2 |-- CMakeLists.txt      顶层 CMake 构建配置
3 |-- README.md          项目说明
4 |-- src/                源代码目录
5 |   |-- main.cpp        入口：读取源文件，依次调用各阶段
6 |   |-- lexer/
7 |   |   |-- Token.h      Token 类型枚举与 Token 结构体定义
8 |   |   |-- Lexer.h/.cpp  词法分析器实现
9 |   |-- parser/
10 |   |   |-- Parser.h/.cpp  递归下降语法分析器
11 |   |   |-- ParseError.h  语法错误类型定义
12 |   |-- ast/
13 |   |   |-- AST.h        全部 AST 节点类（含访问者接口）
14 |   |   |-- ASTPrinter.cpp 调试用：将 AST 打印为树形
15 |   |-- semantic/
16 |   |   |-- SemanticAnalyzer.h/.cpp 语义分析与类型检查
17 |   |   |-- SymbolTable.h 符号表（作用域栈）
18 |   |-- ir/
19 |   |   |-- IRGenerator.h/.cpp  AST -> 三地址码
20 |   |   |-- DAG.h        DAG 结构定义
21 |   |   |-- ThreeAddrCode.h 三地址码指令定义
22 |   |-- optimizer/
23 |   |   |-- Optimizer.h/.cpp  优化 pass 管理器
24 |   |   |-- ConstFold.cpp    常量折叠 pass
25 |   |   |-- CSE.cpp          公共子表达式消除 pass
26 |   |-- codegen/
27 |   |   |-- CodeGen.h/.cpp    代码生成（解释执行或输出汇编）

```

```

28 | |-- utils/
29 |     |-- ErrorHandler.h/.cpp 统一错误报告
30 |-- tests/
31 |     |-- CMakeLists.txt
32 |     |-- test_lexer.cpp
33 |     |-- test_parser.cpp
34 |     |-- test_semantic.cpp
35 |-- examples/          用于测试的示例源文件
36 |     |-- arith.src      纯算术表达式
37 |     |-- control.src    if/while 控制流
38 |     |-- vars.src       变量声明与赋值
    
```

Listing 1: 推荐的项目目录结构

2.1 各模块职责一览

模块	主要职责
lexer/	将字符流切割为 Token，识别关键字、数字、运算符等
parser/	按语法规则将 Token 流构建成 AST
ast/	定义所有语法树节点，提供访问者遍历接口
semantic/	符号表管理、类型检查、变量作用域分析
ir/	将 AST 转换为三地址码或 DAG 中间表示
optimizer/	对 IR 进行各种优化变换（常量折叠、CSE 等）
codegen/	将优化后 IR 转换为目标代码（或解释执行）
utils/	错误报告、位置信息等通用工具

3 辅助工具链

3.1 构建系统：CMake

当项目源文件超过 5 个时，手动编写编译命令极易出错。CMake 是目前 C++ 生态中最主流的构建系统，能自动管理所有 .cpp 文件的编译与链接。

```
1 cmake_minimum_required(VERSION 3.15)
2 project(MyCompiler CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # 收集 src/ 下所有 .cpp 文件
8 file(GLOB_RECURSE SOURCES "src/*.cpp")
9
10 # 生成可执行文件
11 add_executable(mycompiler ${SOURCES})
12 target_include_directories(mycompiler PRIVATE src/)
13
14 # 开启测试
15 enable_testing()
16 add_subdirectory(tests)
```

Listing 2: 顶层 CMakeLists.txt 示例

3.1.1 常用构建命令

```
1 mkdir build
2 cd build
3
4 # 生成构建文件（仅首次需要）
5 cmake ..
6
7 # 编译（-j4 表示 4 线程并行，加快速度）
8 cmake --build . -j4
9
10 # 运行编译器
11 ./mycompiler ../examples/arith.src
12
13 # 运行所有单元测试
```

```
14 ctest --output-on-failure
```

Listing 3: CMake 构建流程（项目根目录执行）

Windows 用户提示

推荐使用 VS Code + CMake Tools 插件，或直接使用 CLion。若在命令行操作，请在“x64 Native Tools Command Prompt”中执行，或安装 MinGW-w64 并将 cmake、g++ 加入 PATH。

3.2 词法/语法生成器：Flex & Bison（可选）

用	自动从 .l 和 .y 文件生成词法/语法分析器的 C/C++ 代码，适合快速原型开发
不用	手写递归下降解析器更灵活、可读性更好，处理复杂错误信息时尤其突出。 大作业推荐手写

3.3 单元测试：Google Test

为每个模块提前写好单元测试，能在集成阶段节省大量排错时间。

```
1 find_package(GTest REQUIRED)
2 include(GoogleTest)
3
4 # 词法分析器测试
5 add_executable(test_lexer test_lexer.cpp)
6 target_link_libraries(test_lexer PRIVATE GTest::gtest_main)
7 target_include_directories(test_lexer PRIVATE ../src)
8 gtest_discover_tests(test_lexer)
```

Listing 4: tests/CMakeLists.txt 示例

3.4 调试工具：GDB

```
1 gdb ./mycompiler
2
3 (gdb) run ../examples/arith.src # 运行程序
```

```

4 (gdb) bt                                # 崩溃后查看调用栈
5 (gdb) frame 2                            # 切换到第 2 层栈帧
6 (gdb) print var_name                    # 打印变量值
7 (gdb) break Lexer.cpp:42                # 在第 42 行设断点
8 (gdb) next                              # 单步（不进入函数）
9 (gdb) step                              # 单步（进入函数）
10 (gdb) quit                             # 退出 GDB

```

Listing 5: GDB 调试速查（先以 Debug 模式编译）

4 编译器各阶段详解

4.1 阶段一：词法分析（Lexer）

4.1.1 任务

词法分析器将源代码的字符流切割为词法单元（Token）流。例如，字符串 "x = 1 + 2;" 会被切割为：

类型（TokenType）	词素（Lexeme）	位置
IDENTIFIER	x	1:1
ASSIGN	=	1:3
NUMBER	1	1:5
PLUS	+	1:7
NUMBER	2	1:9
SEMICOLON	;	1:10
EOF	—	

4.1.2 Token 的设计

每个 Token 建议包含以下字段：

- **type**: 枚举值，区分类型，如 NUMBER、PLUS、IF
- **lexeme**: 对应的原始字符串，如 "123"、"+"

- **line / column**: 行列号，用于生成精确的错误信息
- **value** (可选): 字面量的实际值，如整数 123

4.1.3 实现思路

词法分析器的核心是一个**有限状态机**。维护一个索引指针 `pos` 指向当前字符：

1. 每次调用 `nextToken()`，先跳过空白字符和注释
2. 根据当前字符判断进入哪个“状态”：
 - 数字 0-9: 持续读取，直到遇到非数字，返回 `NUMBER`
 - 字母/下划线: 读取完整标识符，查关键字表；命中则返回关键字 `Token`，否则返回 `IDENTIFIER`
 - 运算符: 单字符直接返回；双字符（如 `<=`）需先 **peek（向前看）** 下一个字符再决定
 - 其他: 报告非法字符错误
3. 关键字识别: 用 `std::unordered_map<string, TokenType>` 存关键字表，查询时间复杂度 $O(1)$

Peek 机制与行列号跟踪

实现 `peek()` 函数——返回下一个字符但不推进指针，用于区分 `<` 和 `<=`、`!` 和 `!=` 等双字符运算符。

行列号跟踪方法：维护 `line` 和 `col` 两个计数器，每读取一个字符 `col++`；遇到 `\n` 时 `line++`，`col = 1`。在创建每个 `Token` 时记录当前的行列号。

4.2 阶段二：语法分析（Parser）

4.2.1 任务

语法分析器依据语言的**语法规则**，将 `Token` 流构建成**抽象语法树（AST）**。

4.2.2 文法定义示例（EBNF）

```

1 program      -> statement*
2 statement    -> varDecl | assignment | ifStmt | whileStmt | block
3 varDecl      -> type IDENTIFIER ('=' expression)? ';'
    
```

```

4 assignment  -> IDENTIFIER '=' expression ';'
5 ifStmt      -> 'if' '(' expression ')' statement ('else' statement)?
6 whileStmt   -> 'while' '(' expression ')' statement
7 block       -> '{' statement* '}'
8
9 expression  -> equality
10 equality    -> comparison ( ('==' | '!=') comparison )*
11 comparison -> addition ( ('<' | '<=' | '>' | '>=') addition )*
12 addition   -> multiply ( ('+' | '-') multiply )*
13 multiply    -> unary ( ('*' | '/') unary )*
14 unary      -> ('-' | '!') unary | primary
15 primary    -> NUMBER | BOOLEAN | IDENTIFIER | '(' expression ')'
    
```

Listing 6: 简单语言的 EBNF 文法

4.2.3 递归下降解析器

为每条语法规则编写一个对应的 C++ 函数，运算符优先级自然编码在函数调用层级中。

运算符优先级的实现原理

优先级越高的运算符，对应的函数调用层级越深。

例如：parseEquality() 调用 parseComparison(), parseComparison() 调用 parseAddition(), parseAddition() 调用 parseMultiply()。

这样 * 和 / 天然比 + 和 - 先被处理，完全符合数学优先级规则，无需额外代码。

4.2.4 错误恢复：Panic Mode（恐慌模式）

遇到语法错误时，不立即退出，而是跳过 Token 直到找到一个“安全同步点”，然后继续解析，以便一次性报告尽可能多的错误。

常用同步点：分号 ;、右花括号 }、关键字 if / while / return。

步骤如下：

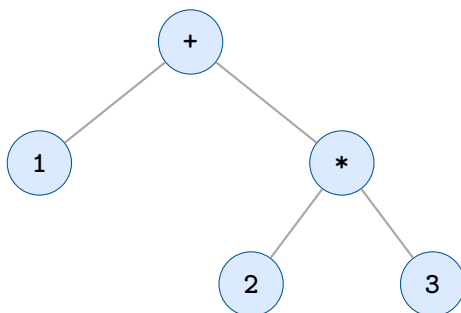
1. 报告当前错误（含行列号）
2. 不断消耗 Token，直到遇到同步点
3. 从同步点继续解析

4.3 阶段三：抽象语法树（AST）

4.3.1 什么是 AST

AST（Abstract Syntax Tree）是源代码的树形中间表示，去掉了括号、分号等语法噪音，只保留程序的语义结构。

以 $1 + 2 * 3$ 为例，其 AST 如下（乘法节点深于加法节点，体现优先级）：



4.3.2 节点类设计（C++ 类层次）

推荐使用继承 + 访问者模式。以下是设计思路（伪代码）：

```

1 // 所有节点的抽象基类
2 struct ASTNode {
3     virtual ~ASTNode() = default;
4     virtual void accept(ASTVisitor& v) = 0;
5 };
6
7 // 表达式基类
8 struct Expr : ASTNode {};
9
10 // 二元运算: left op right
11 struct BinaryExpr : Expr {
12     std::unique_ptr<Expr> left, right;
13     TokenType op;    // PLUS / MINUS / STAR / SLASH ...
14     void accept(ASTVisitor& v) override { v.visit(*this); }
15 };
16
17 // 整数字面量
18 struct NumberLiteral : Expr {
19     int value;

```

```

20     void accept(ASTVisitor& v) override { v.visit(*this); }
21 };
22
23 // 标识符引用（变量名）
24 struct IdentifierExpr : Expr {
25     std::string name;
26     void accept(ASTVisitor& v) override { v.visit(*this); }
27 };
28
29 // 语句基类
30 struct Stmt : ASTNode {};
31
32 // if 语句
33 struct IfStmt : Stmt {
34     std::unique_ptr<Expr> condition;
35     std::unique_ptr<Stmt> thenBranch;
36     std::unique_ptr<Stmt> elseBranch; // 可为 nullptr
37     void accept(ASTVisitor& v) override { v.visit(*this); }
38 };

```

Listing 7: AST 节点基类设计思路

4.3.3 访问者模式（Visitor Pattern）

访问者模式使你可以在不修改节点类的前提下，为 AST 添加新操作（如打印、类型检查、代码生成）：

```

1 struct ASTVisitor {
2     virtual void visit(BinaryExpr& n)      = 0;
3     virtual void visit(NumberLiteral& n)  = 0;
4     virtual void visit(IdentifierExpr& n) = 0;
5     virtual void visit(IfStmt& n)         = 0;
6     // ...后续添加更多节点类型时，在此新增虚函数
7 };
8
9 // 实现示例：AST 打印机（调试用）
10 struct ASTPrinter : ASTVisitor {
11     int depth = 0;
12     void visit(BinaryExpr& n) override {

```

```

13     // 打印缩进 + 运算符，然后递归打印左右子树
14 }
15 // ...
16 };
    
```

Listing 8: 访问者接口定义

unique_ptr 与所有权

AST 中父节点拥有子节点的所有权。使用 `std::unique_ptr` 来表示，父节点销毁时子节点自动递归销毁，彻底杜绝内存泄漏，无需手动 `delete`。

4.4 阶段四：语义分析

4.4.1 任务

语法分析验证代码**结构**是否合法，语义分析验证代码**含义**是否合法：

- 变量必须先声明再使用（未声明应报错）
- 赋值时右值类型必须与左值兼容（类型检查）
- 函数调用参数数量与声明一致（参数检查）

4.4.2 符号表设计

符号表记录所有标识符的名称、类型、作用域层级等信息。

```

1 struct Symbol {
2     std::string name;
3     DataType   type;        // INT, BOOL, FLOAT ...
4     int         scopeLevel;
5 };
6
7 class SymbolTable {
8     // 用栈模拟作用域嵌套：最内层作用域在栈顶
9     std::vector<std::unordered_map<std::string, Symbol>> scopes;
10 public:
11     void enterScope();        // 遇到 '{' 时调用
12     void exitScope();         // 遇到 '}' 时调用
13     void define(const Symbol& s);
    
```

```

14 Symbol* lookup(const std::string& name);
15 // lookup 从栈顶向下查找，实现"内层屏蔽外层"
16 };
    
```

Listing 9: 符号表结构示意

4.4.3 类型检查

遍历 AST，对每个表达式推导其类型并验证操作数兼容性：

操作	合法性	结果类型
<code>int + int</code>	合法	<code>int</code>
<code>int < int</code>	合法	<code>bool</code>
<code>int + bool</code>	非法（报错）	—
<code>int = bool</code>	非法（报错）	—

4.5 阶段五：中间代码生成（IR）

4.5.1 为什么需要中间表示

直接从 AST 生成目标代码不便于优化。IR（Intermediate Representation）介于 AST 和目标代码之间，是方便进行各种优化分析的统一表示形式。

4.5.2 三地址码（Three-Address Code）

每条指令最多含三个地址（两操作数、一结果）：

```

1 x = y op z      // 二元运算（加减乘除、比较）
2 x = op y        // 一元运算（负号、逻辑非）
3 x = y           // 赋值/复制
4 goto L          // 无条件跳转
5 if x goto L     // 条件跳转
6 call f, n       // 函数调用（n 为参数数量）
    
```

Listing 10: 三地址码指令类型示例

生成思路：对 AST 进行**后序遍历**（先处理子树，再处理根节点），每处理一个节点就生成对应指令，并用临时变量 t1、t2... 存储中间结果。

以 if (x > 0) y = 1; else y = -1; 为例：

```

1      t1 = x > 0
2      if t1 == false goto L_ELSE
3      y = 1
4      goto L_END
5 L_ELSE:
6      t2 = -1
7      y = t2
8 L_END:
    
```

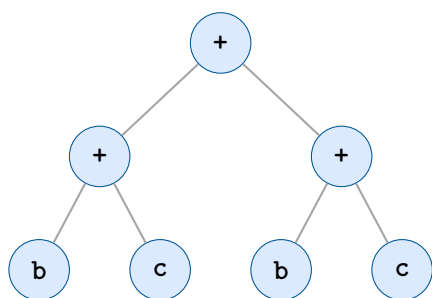
Listing 11: if-else 语句的三地址码生成示例

4.5.3 DAG（有向无环图）

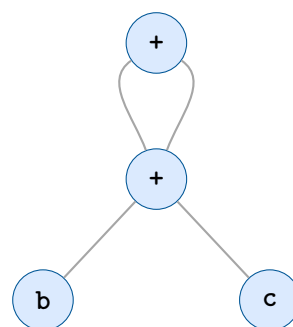
DAG（Directed Acyclic Graph）允许多个节点**共享相同的子树**，从而天然消除重复计算。

考虑 $a = (b + c) + (b + c)$ ，AST 中 $b+c$ 出现两次，而 DAG 中根节点的两条边指向**同一个 $b+c$ 节点**：

AST（有冗余）



DAG（共享子节点）



DAG 构建算法核心

维护一个哈希表，键为“运算符 + 左操作数 ID + 右操作数 ID”。创建新节点前先查表：若已存在相同节点，直接返回已有 ID（复用），否则创建新节点并插入表。这就是公共子表达式消除（CSE）的本质。

4.6 阶段六：代码生成

4.6.1 目标选择（由易到难）

1. **解释执行三地址码**（最简单，强烈推荐入门）：用 C++ 模拟一个虚拟机，直接执行三地址码指令，可验证编译器前几阶段的正确性。
2. **生成 LLVM IR**（推荐进阶）：调用 LLVM C++ API，由 LLVM 负责后端优化和平台汇编生成。
3. **生成 x86-64 汇编**（最复杂）：直接输出 .asm 文件，需要了解寄存器分配等底层知识。

关于寄存器分配

真实的寄存器分配（图着色算法）相当复杂。对于大学生项目，可以使用**简化策略**：将所有临时变量存放在栈帧上（通过 `mov` 指令存取内存），性能不高但实现简单、功能完全正确。等功能稳定后再考虑优化寄存器使用策略。

5 完整构建步骤

5.1 开发环境准备

1. 安装 C++ 编译器：
 - Windows: 安装 MinGW-w64, 或 Visual Studio (含 MSVC)
 - Linux: `sudo apt install build-essential cmake`
 - macOS: `xcode-select --install`
2. 安装 CMake 3.15+: 从 cmake.org 下载
3. 配置编辑器: VS Code + C/C++ 扩展 + CMake Tools 扩展
4. (可选) 安装 Google Test: 通过 `vcpkg` 执行 `vcpkg install gtest`

5.2 推荐开发顺序

按此顺序开发，不容易迷失方向

1. 词法分析器：最容易入手，先让 "1 + 2" 正确输出 Token 序列
2. 语法分析器 + AST：实现表达式解析，能打印 AST 的树形结构
3. 语义分析：加入符号表，能检测未声明变量和类型不匹配
4. 解释执行（可选）：直接遍历 AST 求值，验证前三阶段正确性
5. IR 生成：将 AST 转换为三地址码
6. 基础优化：在 IR 上做常量折叠等简单优化
7. 代码生成：解释执行 IR 或输出目标代码

5.3 增量验证策略

每完成一个阶段，立即用简单案例验证正确性，不要等到最后才测试：

阶段	验证方式
词法	对 "x = 1 + 2 * 3;" 打印出正确的 Token 序列
语法	对上述输入打印出正确的 AST 树形
语义	对未声明变量 y 报错；对 int x = true; 报类型错误
IR	对 "a = 1 + 2 * 3;" 生成正确的三地址码
代码	对 "int a=1; int b=2; print(a+b);" 输出 3

6 性能优化建议

优化的黄金原则

先让编译器正确运行，再做优化！优化应在 IR 阶段以独立的“pass”形式实现，不要在 AST 阶段修改树结构。每个 pass 只做一件事，便于单独测试和调试。

6.1 常量折叠 (Constant Folding)

若表达式的所有操作数在编译期都是常量，则直接计算结果，不生成任何运行时计算指令。

优化前		优化后
<code>1 + 2</code>	$\Rightarrow$	<code>3</code>
<code>3 * 4 - 2</code>	$\Rightarrow$	<code>10</code>
<code>2 * 0</code>	$\Rightarrow$	<code>0</code>
<code>true && false</code>	$\Rightarrow$	<code>false</code>

实现思路：在生成 IR 时，若 BinaryExpr 的两个子节点均为字面量，则在编译期直接计算结果，返回一个新的字面量节点，不生成三地址码指令。

6.2 代数化简 (Algebraic Simplification)

利用基本数学性质化简表达式，实现简单但收益明显：

原表达式	规则	化简结果
<code>x + 0</code>	加法单位元	<code>x</code>
<code>x * 1</code>	乘法单位元	<code>x</code>
<code>x * 0</code>	零乘法则	<code>0</code>
<code>x - x</code>	自身相减	<code>0</code>
<code>x / x</code>	自身相除	<code>1</code> (排除 <code>x=0</code>)
<code>-(-x)</code>	双重否定	<code>x</code>
<code>x + (-y)</code>	加负等于减	<code>x - y</code>

实现思路：在 IR 优化 pass 中，对每条三地址码做模式匹配，识别并替换符合上述规则的指令。例如，识别 $x = y + 0$ 直接替换为 $x = y$ 。

6.3 负号合并详细示例

以 $1 + (-1)$ 为例，展示代数化简的完整优化过程：

```

1 // 源代码
2 a = 1 + (-1)
3
4 // 未优化的三地址码（直接翻译）
5 t1 = -1           // 一元负号：生成一条指令
6 t2 = 1 + t1       // 加法：又生成一条指令
7
8 // 第一步：代数化简（加负 -> 减法）
9 // 识别 "x + (-y)" 模式，改写为 "x - y"
10 t2 = 1 - 1       // 省去了 t1 的计算，少一条指令
11
12 // 第二步：常量折叠（两个常量相减）
13 a = 0            // 直接得到结果，彻底消除运算
    
```

Listing 12: $1 + (-1)$ 的分步优化过程

6.4 公共子表达式消除（CSE）

若同一个表达式在程序中被计算多次，只计算一次并将结果复用：

```

1 // 优化前的三地址码
2 t1 = b + c
3 t2 = b + c       // 与 t1 完全相同，重复计算！
4 t3 = t1 * t2
5
6 // CSE 优化后
7 t1 = b + c
8 t3 = t1 * t1     // 复用 t1，省去一次加法
    
```

Listing 13: CSE 优化示例

实现思路：

- **方法一（推荐）：**基于 DAG 构建（见第 4.5 节），哈希表机制天然支持 CSE
- **方法二：**在三地址码上维护“可用表达式集合”，每次生成新指令前查表，若已有相同表达式则复用结果变量名

6.5 死代码消除（Dead Code Elimination）

删除不可能被执行、或对最终结果没有影响的代码：

```

1 // 优化前（if 条件是编译期常量 false）
2 if (false) {
3     x = 100;    // 永远不会执行
4 }
5 y = 2;
6
7 // 死代码消除后（整个 if 分支被删除）
8 y = 2;
```

Listing 14: 死代码消除示例

实现思路：在常量折叠之后，检查 if 语句的条件是否已经是编译期常量(true/false)，若是则只保留对应分支，丢弃另一分支代码。

6.6 指令重排（Instruction Reordering）

在不改变程序语义的前提下，调整指令顺序以减少流水线停顿、提高 CPU 利用率：

```

1 // 重排前（存在数据依赖导致流水线停顿）
2 t1 = load a        // 访问内存，有延迟
3 t2 = t1 + 1        // 必须等 t1 就绪，流水线空转
4 t3 = load b        // 独立操作，可以提前
5
6 // 重排后（t3 的加载与 t1 的加载并行进行）
7 t3 = load b        // 提前执行，填充流水线空泡
8 t1 = load a
9 t2 = t1 + 1        // 此时 t1 已就绪
```

Listing 15: 指令重排示例（减少数据依赖等待）

何时可以重排

两条指令可以交换顺序，当且仅当：

1. 它们之间没有数据依赖（第一条的输出不是第二条的输入）
2. 没有副作用冲突（如不写同一内存地址）

对大学生项目来说，实现一个简单的“检查两条指令是否读写相同变量”的判断函数即可满足基本需求。

7 实战难点预警

7.1 内存管理

黄金法则：所有权明确

在 AST 和 IR 中，始终用 `std::unique_ptr` 表示所有权。禁止裸 `new/delete`，让 RAII 自动管理生命周期。

传递规则：

- 转移所有权 → 用 `std::move(ptr)`
- 只是借用，不转移 → 传 `T*` 裸指针或 `const T&`

可以用 `using` 别名减少视觉负担：

```

1 // 在 AST.h 开头定义
2 template<typename T>
3 using Ptr = std::unique_ptr<T>;
4
5 // 使用时更简洁
6 Ptr<Expr> left;    // 等同于 std::unique_ptr<Expr> left;
7 Ptr<Stmt> body;    // 等同于 std::unique_ptr<Stmt> body;
```

Listing 16: 用 `using` 简化智能指针书写

7.2 统一错误处理

设计 `ErrorHandler` 类，让所有模块通过它报告错误，而不是直接打印到 `std::cerr`：

```

1 class ErrorHandler {
```

```

2   int errorCount    = 0;
3   int warningCount = 0;
4 public:
5   void error(int line, int col, const std::string& msg) {
6       ++errorCount;
7       std::cerr << "error: [第" << line << "行, 第"
8           << col << "列] " << msg << "\n";
9   }
10  void warning(int line, int col, const std::string& msg);
11  bool hasErrors() const { return errorCount > 0; }
12 };

```

Listing 17: ErrorHandler 设计思路

好的错误信息示例:

```

1 error:   [第5行, 第10列] 期望 ';' 但遇到了 'if'
2 error:   [第8行, 第3列]  变量 'x' 未声明即使用
3 warning: [第3行, 第7列]  变量 'tmp' 已声明但从未使用

```

Listing 18: 错误信息格式参考

7.3 常见 Bug 预警

这些 Bug 几乎人人都会遇到

1. 无限循环解析: 递归下降中忘记消耗 Token, 导致无限递归直至栈溢出。排查方法: 每次进入解析函数, 确保 Token 指针在某处前进了。
2. 符号表作用域泄漏: 退出 } 时忘记调用 exitScope(), 导致内层局部变量在外层仍然可见。
3. 移动后再使用 unique_ptr: 将 unique_ptr 通过 std::move() 传入函数后, 原变量变为 nullptr, 再次解引用会直接崩溃。
4. 空 else 分支: if 不一定有 else, 在代码生成阶段必须检查 elseBranch != nullptr。
5. 左递归死循环: 文法中存在左递归 (如 `expr -> expr '+' expr`) 会使递归下降解析器无限循环, 需改写为右递归或迭代 (循环) 形式。

7.4 推荐学习资源

- **Crafting Interpreters** (Robert Nystrom, 免费在线): 用 Java/C 从零实现解释器, 思路极为清晰, **强烈推荐作为主要参考**
- 《自制编译器》(青木峰郎): 中文版, 适合大学生入门, 示例代码清晰易懂
- 《编译原理》(龙书, Aho 等): 权威教材, 内容全面, 可作为查阅参考
- **LLVM Kaleidoscope Tutorial:**
<https://llvm.org/docs/tutorial/>
从零用 LLVM API 实现一门完整语言, 是学习 LLVM 后端的最佳起点

8 总结

构建编译器是计算机科学中最有成就感的项目之一。牢记以下要点:

1. **一步一步来:** 先跑通 `1 + 2`, 再扩展 `if/while`, 不要一开始就沉迷于复杂优化
2. **打印中间结果:** 词法阶段打印 Token, 语法阶段打印 AST, 这是定位 bug 最快的方法
3. **测试先行:** 每完成一个模块立即写测试, 不要等到集成阶段再测
4. **用好工具:** CMake 管理构建, GDB 调试崩溃, Git 保存进度
5. **不怕出错:** 编译器的错误信息是你的朋友, 认真阅读每一条

祝你的编译器大作业顺利完成!